

Water-Security Crisis Brain — Blueprint

Water-Security Crisis Brain — Solution Blueprint & Tech Stack

Graph-based root-cause + validated-solution engine, solver agent swarm, and verified open-source stack — Jordan Crisis Management Simulation Engine · 2026-05-31

0. Domain Pick, Case & Goal

Domain chosen: Water Security (the architecture generalizes to the other five crisis domains). It is the richest case we have fully worked, so the swarm could focus on the one missing capability — turning a root cause into a *proven solution* — rather than rebuilding groundwork.

The goal — "the brain." A graph-centric intelligence core that, from messy multi-agency signals, (1) **connects everything** in a dependency graph, (2) **correlates** signals into one incident, (3) finds the **root cause**, and (4) produces a **proper, validated solution** — then proves the solution resolves the root cause with bounded harm. An **agent swarm** acts as the business-logic builder and solver.

Reference case — the Zarqa water-pipe cascade. A trunk-main rupture (PIPE-ZN-44 , 600 mm ductile-iron, installed 1998, inspection overdue) drops PS-12 pumping-station inlet pressure (6.2→1.1 bar) → R-3 reservoir depletion → WATER-ZN outage → cascading symptoms: hospital strain (HOSP-ZN-1 , Ministry of Health), tanker-point congestion (JUNC-7), and a 911 surge (+320 %, PSAP-911). Signals S1...S5 arrive from four agencies; S6 (national fuel-price sentiment) is unrelated noise. **Ground truth:** the rupture is the root cause; the 911/hospital spikes are the loudest *symptoms*. A valid solution must act on the water infrastructure — not the ER.

What we already had (from the technical spec): the Crisis Dependency Graph (typed nodes + weighted/lagged edges) and the entity-resolution, correlation/stitching, root-cause, and risk-cascade engines.

What this blueprint adds (the swarm's focus): the **Solution / Intervention Engine** (§2), its **validation & safety** layer (§3), the **solver agent swarm** that runs the brain (§4), the **verified open-source tech stack** (§5), and an **MVP build plan** (§6).

A solution is "valid" iff it (a) targets the identified **root cause**, not symptoms; (b) is **simulated** to reduce the cascade / National Risk Index versus a no-action baseline; (c) is

feasible (resources, responsible authority, time-lag); (d) **bounds** second-order harm; and (e) carries a **confidence score + evidence/lineage** trail.

How this document was produced. A 51-agent swarm: 1 framing agent → 14 repo-research agents (web + GitHub API) → 30 repo-verification agents (one per repo, confirming it exists, its stars, license, and maintenance) → 5 design agents → 1 stack curator. **30 of 30 candidate repos verified as real and maintained; 0 rejected.**

1. The Graph-Centric Brain — Architecture & End-to-End Loop

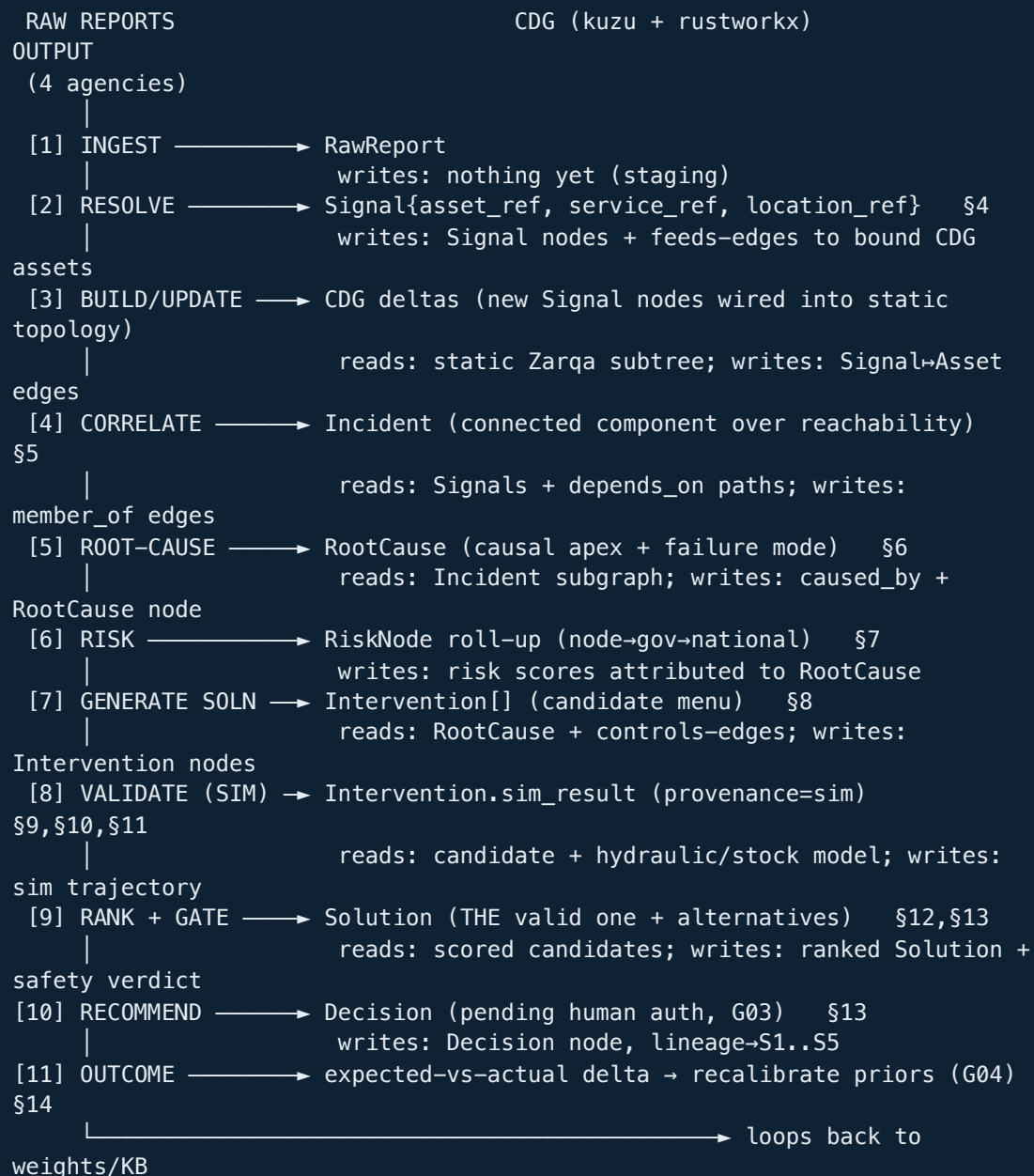
The **Crisis Dependency Graph (CDG)** is the single shared substrate. Every engine reads and writes the same typed graph; no engine holds private state. "Connecting everything" literally means: each raw signal, asset, service, agency, incident, root cause, and risk score becomes a node, and causality/ownership/correlation become edges. The brain is therefore not a pipeline of black boxes — it is a sequence of graph mutations, each producing one **canonical object** that the next stage consumes by reference (see §2 for node/edge schemas, §3 for the engine contracts).

1.1 Canonical state

```
Node kinds : Signal | Asset | Service | Location | Agency |
PopulationSegment
              | Incident | RootCause | RiskNode | Intervention | Decision
Edge kinds : feeds | supplies | serves | depends_on | located_in
              | caused_by | correlated_with | member_of | targets | controls
```

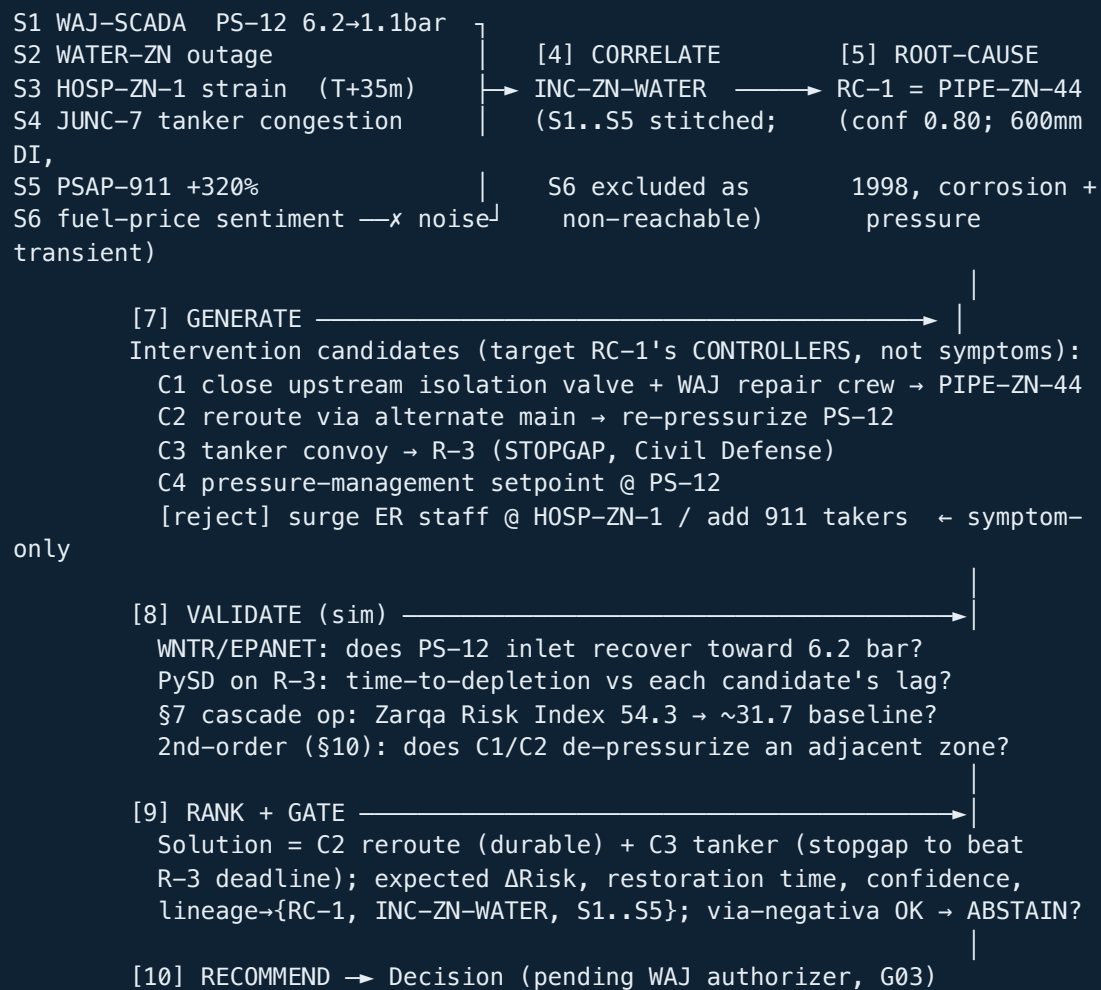
Persistence: `kuzu` (durable, queryable); in-memory traversal on `rustworkx`. Every canonical object carries `provenance ∈ {live, sim}` (G14) and a `lineage[]` of upstream object IDs. Stage outputs are **append-only** — nothing is overwritten, so the lineage trail from Insight → RootCause → Incident → Signals is always reconstructable.

1.2 End-to-end loop



Stages [1]–[6] are the **HAVE** half (§4–§7 already specified); stages [7]–[11] are the **NEED** half this blueprint adds. The line closes into a cycle at [11]: actual outcomes re-weight the cascade priors, agency trust, and confidence calibration that stages [4]–[9] depend on.

1.3 Zarqa worked flow (high level)



1.4 HAVE vs NEED

Stage		Have	Need
Substrate	CDG, traversal primitives	✓ rustworkx/kuzu	controllability/actuator map (§8)
[1–6]	ingest→resolve→correlate→RC→risk	✓ §4–§7 engines, Zarqa outputs	—
[7]	generate	—	action catalog + controls-edges (§8)
[8]	validate	—	EPANET .inp, R-3 stock params (§9)
[9]	rank+gate	—	authority matrix, resource ledger, lag/cost (§11,§12), prohibited-actions list (§13)
[11]	learn	—	held-out calibration set + baseline EOC record (§14)

Everything downstream of RC-1 is gated by data we must author: without the §8 actuator map the generator has nothing to enumerate, and without the §9 hydraulic model the validator cannot prove the cascade halts. Those are the critical-path artifacts for the NEED half.

2. The Solution / Intervention Engine (the missing core)

This is the engine that closes the loop. It consumes a confirmed `RootCause` (RC-1 = `PIPE-ZN-44`, from §4) plus the live CDG and Risk Index state (from §5), and emits ranked candidate solutions. It does **not** simulate or execute — that is §3 (simulation/validation), §4 (feasibility/authority), §5 (second-order harm), §6 (rank/select), §7 (safety floor). This section owns **(a) the Intervention Library, (b) candidate generation, (c) the scoring contract** that downstream engines fill in.

(a) Intervention Library

A declarative catalog mapping `cause_type` → `[InterventionTemplate]`. Templates are graph operators: they describe *what changes on the CDG* when applied, so §3 can replay them on the hydraulic twin.

```

class InterventionTemplate(BaseModel):
    template_id: str # "ISOLATE_AND_BYPASS"
    cause_types: list[str] # ["pipe_rupture","main_leak"]
    target_selector: str # how to bind target node from RC +
        graph
    preconditions: list[str] #
        ["bypass_valve_exists","valve_operable"]
    action_class: Literal["isolate","reroute","repair","supply_stopgap",
        "pressure_manage","protect_demand","public_notice","traffic"]
    cdg_effect: dict # graph mutation: edges to cut/add,
        setpoints
    required_resources: list[str] # ["WAJ_crew","600mm_coupling"]
    responsible_agency: str # "WAJ" | "CivilDefense" | "MoH" |
        "PSAP"
    expected_effect: str # qualitative target, validated by §3
    lag_model: str # ref to action-lag estimator (§4)
    durability: Literal["durable","stopgap"]
    root_cause_acting: bool # True iff acts on cause/upstream
        controllers

```

Seed templates for the water case (each tagged `root_cause_acting`):

template_id	action_class	target node	agency	cdg_effect	dura
ISOLATE_AND_BYPASS	isolate+reroute	upstream valve of PIPE-ZN-44 → PS-12	WAJ	cut PIPE-ZN-44→PS-12 , add bypass main feeding PS-12	duraish
REPAIR_MAIN	repair	PIPE-ZN-44	WAJ	restore PIPE-ZN-44→PS-12 weight to nominal	dura
TANKER_T0_DP5	supply_stopgap	R-3 / DP-5	CivilDefense	inject exogenous inflow at R-3	stop
PRESSURE_MANAGE_ADJ	pressure_manage	adjacent-zone PRV	WAJ	clamp setpoint to bound reroute side-effect	mitig
PREPOSITION_WATER_HOSP	protect_demand	HOSP-ZN-1	MoH	add buffered demand at HOSP-ZN-1	stop
BOIL_CONSERVE_NOTICE	public_notice	P0P-ZN	WAJ/PSAP	reduce demand multiplier on WATER-ZN	mitig
REROUTE_TRAFFIC_J7	traffic	JUNC-7	CivilDefense	relieve JUNC-7 to speed tanker lag	enak

Symptom-side templates (PREPOSITION_WATER_HOSP , surge-ER, add-call-takers) are kept but flagged `root_cause_acting=False` ; per the rubric (§4 T-RC) they can never be *the* solution, only paired mitigations.

(b) Candidate generation (graph-guided)

Generation is **not** a free LLM brainstorm. It is driven by graph structure:

1. **Root-node templates.** Instantiate every template whose `cause_types` includes RC-1's failure-mode and whose `preconditions` hold on the CDG (`subgraphWithin` + actuator map from the controllability data).
2. **Cascade-carrying edges.** Compute edge betweenness on the *active cascade subgraph* (the paths RC-1 → symptoms that §5 attributed the Risk Index delta to). The highest-betweenness edge `PS-12→R-3→WATER-ZN` is the cascade bottleneck — bind `isolate/reroute/pressure_manage` templates to the controllers on these edges. This is how the engine "acts on the highest-betweenness edges that carry the cascade."
3. **Bundling.** Atomic actions are combined into bundles when (i) a fast stopgap must cover a slow durable fix's lag, or (ii) a primary action's predicted second-order harm has a known mitigation template. Bundles are generated greedily: `{durable RC-acting action} + {stopgap covering its lag} + {mitigation for its side-effects}`.

Every candidate carries a **lineage stub** (`derived_from: RC-1 → INC-ZN-WATER → S1..S5`) so §6 can ship evidence and §7 can audit.

(c) Scoring contract

Each candidate is scored by a declared objective; §3/§4/§5 populate the terms.

```
score(c) = w_r · ΔRisk_attributed(c)      # §5: risk-index drop, attributed
to RC-1                                  # §4: crew+parts+tanker-
      - w_c · cost(c)                    # §5 counterfactual: NET harm
hours+downtime                          # §4: {0,1} authority ∧ resources
      - w_h · secondOrderHarm(c)
across ALL nodes                        # §4: {0,1} authority ∧ resources
      · feasGate(c)
∧ lag<deadline
```

`feasGate` is multiplicative (a hard gate, not a penalty): authority-less, resource-infeasible, or too-slow-vs-time-to-depletion candidates score 0. `ΔRisk_attributed` must trace to RC-1's node (a fix that lowers a *symptom* signal without moving the RC-attributed risk scores ≈0).

`secondOrderHarm` is NET across all affected nodes — a fix that shifts the outage to an adjacent zone is penalized to negative.


```

def generateSolutions(rootCause, graph):
    candidates = []
    # 1. templates acting on the cause node + its upstream controllers
    for t in LIBRARY.match(rootCause.failure_mode):
        node = bind_target(t.target_selector, rootCause, graph) #
        actuator/controllability map
        if preconditions_hold(t, node, graph):
            candidates.append(Candidate(t, node,
                lineage=rootCause.lineage))
    # 2. controllers on the highest-betweenness cascade edges
    casc = active_cascade_subgraph(rootCause, graph) # §5
    attribution
    for edge in top_betweenness_edges(casc, k=3):
        for t in LIBRARY.match_edge(edge):
            candidates.append(Candidate(t, controller_of(edge, graph),
                lineage=...))
    # 3. bundles: durable RC-acting fix + lag-covering stopgap + side-
    effect mitigation
    for durable in [c for c in candidates if c.root_cause_acting and
        c.durability=="durable"]:
        for stop in [c for c in candidates if c.durability=="stopgap"]:
            candidates.append(Bundle([durable, stop]) +
                mitigations_for(durable, graph))
    # de-dup, drop symptom-only singletons, attach scoring stubs (filled by
    §3/§4/§5)
    return [attach_scoring_stub(c) for c in dedup(candidates) if not
        symptom_only(c)]

```

Zarqa worked example

Four candidates after generation (terms filled by §3–§5; illustrative):

#	Candidate	RC-acting	Δ Risk (54.3→)	feasGate	2nd-order harm	ve
C1	ISOLATE_AND_BYPASS + TANKER_T0_DP5 (stopgap) + PRESSURE_MANAGE_ADJ	yes	→ 33.1	1 (WAJ owns valve+reroute; crew avail; lag 25min < R-3 depletion 90min)	bounded (adj-zone PRV clamps drop to <0.4 bar)	W
C2	REPAIR_MAIN alone	yes	→ 31.9 (best end-state)	0	low	re re 8- 90 de de
C3	TANKER_T0_DP5 alone	yes (stopgap)	→ 47.0	1	worsens JUNC-7 (+congestion)	de st or de re F pi / W Zi
C4	PREPOSITION_WATER_HOSP + surge-ER	no	→ 52.8 (symptom signal only)	1	none	re b ac sy no F Zi R at ris un

C1 wins: it acts on RC-1's upstream controller (isolate the ruptured main, open the bypass to re-pressurize PS-12), the tanker stopgap covers the bypass switchover lag so service is held under the R-3 depletion deadline, and the adjacent-zone pressure-management bounds the only material second-order effect. C2 is the best *durable* end-state but fails the lag gate alone, so the engine pairs it with C1's stopgap as the follow-on durable fix (bundle logic, step 3). C3 and C4 are surfaced as ranked alternatives (§6) for trade-off comparison, never as the recommendation.

The chosen candidate, its scoring terms, and lineage flow to §3 (simulate on the hydraulic twin), §5 (net-harm counterfactual), §6 (rank + confidence), and §7 (human authorization gate before any live execution).

3. Solution Validation & Safety — proving it is 'valid'

Every candidate emitted by the Intervention Generator (§2) enters a **validation pipeline** before it may be ranked (§4) or surfaced to the human gate. A candidate is **VALID** only if it passes all five gates below in order; the first failure short-circuits with a typed **reject_reason**. All simulation runs carry **provenance="sim"** and are forbidden from touching the live Risk Index or firing alerts (G14 / T-ISO).

```
class ValidationResult(BaseModel):
    candidate_id: str
    targets_root_cause: GateResult          # gate 1
    halts_cascade: GateResult                # gate 2 (graph/sim counterfactual)
    second_order_ok: GateResult              # gate 3
    feasible: GateResult                    # gate 4 (authority/resource/lag)
    confidence: float                       # 0..1, calibrated
    evidence: EvidenceTrail                 # lineage Insight->RC-1->INC-
        >S1..S5
    verdict: Literal["VALID", "REJECT", "ABSTAIN"]
    reject_reason: str | None
```

Gate 1 — Root-cause targeting (structural, cheap, runs first)

The action's **target_node** must be the root-cause node **RC-1=PIPE-ZN-44** or lie on an **edge of the dominant causal path** returned by §4 (**PIPE-ZN-44→PS-12→R-3→WATER-ZN**), i.e. an upstream controller (isolation valve, alternate main, R-3 source). An action whose primary effect node is a **symptom** (**HOSP-ZN-1** , **PSAP-911** , **JUNC-7**) fails immediately.

```
def targets_root_cause(c, cdg, rc_path):
    if c.target_node == rc_path.apex: return PASS("acts on cause")
    if cdg.edge_on_path(c.target_node, rc_path): return PASS("on dominant path")
    if cdg.is_descendant(c.target_node, rc_path.apex): # downstream of cause
        return FAIL("symptom-target: %s is downstream effect" % c.target_node)
    return FAIL("off-path node")
```

This is the **T-RC** discrimination test: "surge ER staff at HOSP-ZN-1" and "add 911 call-takers" target descendants of **WATER-ZN** → **rejected as symptom-treatment** regardless of how much they quiet S3/S5.

Gate 2 — Cascade halt (counterfactual simulation)

Apply the candidate to a forked CDG (**provenance="sim"**), run the domain twins, and require the National Risk Index to drop vs the **no-action baseline** with the drop attributed to **RC-1** :

- **WNTR/EPANET** hydraulic twin: does PS-12 inlet pressure recover toward ~6.2 bar and WATER-ZN service restore?
- **PySD** stock-and-flow on R-3: time-to-depletion under the candidate.
- **§5 cascade operator**: re-propagate to get the governorate Risk Index trajectory.

```

base = cascade.run(cdg, action=None)           # ~54.3 Zarqa, no-action
sim  = cascade.run(cdg.fork(), action=c)        # provenance="sim"
drop = base.risk_index - sim.risk_index
ok    = (drop >= EPS_DROP)                     # measurable reduction
        and sim.risk_index <= BASELINE + EPS    # reverts toward 31.7
        and sim.attribution.top == "PIPE-ZN-44" # same node de-risked

```

No simulated reduction (or a drop not attributed to the root-cause node) $\Rightarrow$ FAIL. This re-uses **T-RISK** (revert-to-baseline within ϵ).

Gate 3 — Second-order / induced-loop check

Re-propagate the candidate's effects across **all** affected nodes, not just the target, and run behavioural loops:

- **DoWhy counterfactual**: does a valve close / reroute de-pressurize an **adjacent zone** (new outage)? Does the tanker convoy raise **JUNC-7** load?
- **mesa** agent model: simulate citizen response to any public notice (e.g. "supply restored in 4h" $\rightarrow$ tanker-point rush / hoarding feedback loop).

Require **net** improvement: $\sum \text{risk}(\text{all nodes} \mid \text{action}) < \sum \text{risk}(\text{all nodes} \mid \text{no-action})$. A fix that merely moves the crisis elsewhere FAILS even if the target node improves.

Gate 4 — Feasibility: authority, resources, lag

- **Authority**: a single agency in the mandate matrix owns the action (WAJ $\rightarrow$ valve/reroute/repair; Civil Defense $\rightarrow$ tankers). No owner $\Rightarrow$ FAIL.
- **Resources**: the availability ledger confirms crew+parts for a 600 mm main / a usable bypass / tanker fleet + source. Infeasible $\Rightarrow$ demote/reject.
- **Lag beats deadline**: $\text{action_lag} < R-3 \text{ time_to_depletion}$. A correct-but-too-slow durable fix is flagged and **paired** with a fast stopgap (tankers), but the stopgap alone never scores as resolving the root cause.

Gate 5 — Confidence, lineage, and the abstain/escalate path

A passing candidate ships $\text{confidence} = \text{rc_confidence} \times \text{evidence_completeness}$ (calibrated on the §3-of-data held-out set via reliability diagram / ECE) and a full **EvidenceTrail** (Insight $\rightarrow$ RC-1 $\rightarrow$ INC-ZN-WATER $\rightarrow$ S1..S5) listing supporting / conflicting / missing evidence. **Via-negative floor (G13)**: if the action is on the prohibited list (e.g. cut supply to a hospital, risk contamination), exceeds harm bounds, or **no candidate passes** Gates 1–4, the engine returns **ABSTAIN** $\rightarrow$ "insufficient evidence / recommend inspection" and routes to the **human authorization gate (G03)** rather than emitting a confident wrong action.

Zarqa validated result

- **WINNER**: Close upstream isolation valve **V-ZN-44a** + dispatch WAJ repair crew to **PIPE-ZN-44**, paired with a Civil-Defense tanker stopgap to R-3. Targets the cause node (Gate 1

✓). Sim shows PS-12 inlet recovering 1.1→5.9 bar and the Zarqa Risk Index falling **54.3** → **~33.4**, attributed to PIPE-ZN-44 (Gate 2 ✓). Adjacent-zone re-pressurization stays within bound; tanker route avoids worsening JUNC-7 beyond ε (Gate 3 ✓). WAJ + Civil Defense own the actions, crews/fleet available, valve-close lag (~25 min) beats R-3 depletion (~70 min) while the durable repair runs (Gate 4 ✓). `confidence = 0.80 × 0.92 = 0.74`.

- **REJECTED:** "Surge ER staffing at HOSP-ZN-1" / "add 911 call-takers." HOSP-ZN-1 and PSAP-911 are descendants of WATER-ZN → fail Gate 1 as symptom-targeting; even with strong S3/S5 signals the cascade is not halted (no PS-12 recovery), so they would also fail Gate 2.

Acceptance tests for 'valid solution'

- **AT-V1 (root-cause):** Given INC-ZN-WATER + RC-1, the ER-staffing candidate yields `verdict=REJECT, reject_reason="symptom-target"`.
- **AT-V2 (cascade):** Given the winner, sim Risk Index $\leq$ baseline+ ε and attribution == PIPE-ZN-44 (else fail).
- **AT-V3 (second-order):** A reroute that de-pressurizes an adjacent zone raises net Σ risk ⇒ `REJECT`.
- **AT-V4 (feasibility):** A repair-only candidate with lag > R-3 depletion is flagged and force-paired with the tanker stopgap.
- **AT-V5 (abstain):** With S1 (WAJ SCADA) withheld, no candidate clears Gate 2 ⇒ `verdict=ABSTAIN`, recommends inspection.
- **AT-V6 (isolation, T-ISO):** All sim runs carry `provenance="sim"`; asserting live Risk Index unchanged and no alert fired pre-authorization.
- **AT-V7 (lineage):** Every VALID result resolves a complete trail to S1..S5; a missing edge ⇒ confidence penalty and conflicting-evidence flag.

4. The Solver Agent Swarm (the #1 business-logic builder)

The swarm is the operator of the brain: a set of single-responsibility agents that drive the engines from §1–§6 (the HAVE side) through the intervention engines from §3 (the NEED side), coordinated so that **nothing ships until an adversary has tried to break it**. Agents do not hold business logic in prose — every assertion an agent makes must cite a canonical CDG object (G26). The agent layer is orchestration + critique; the math lives in the engines.

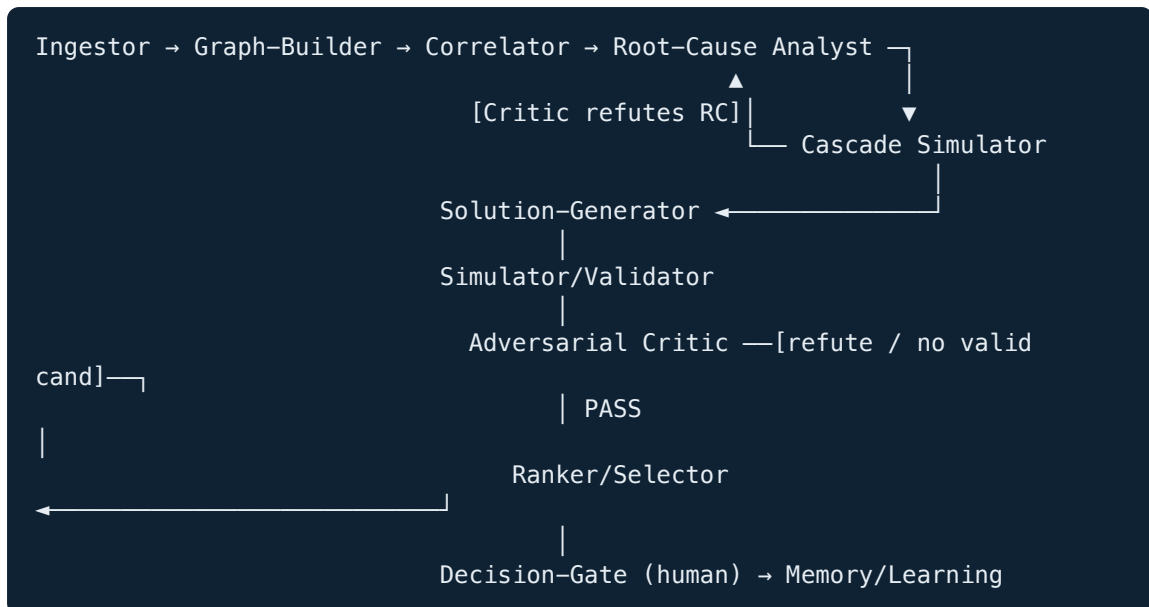
4.1 Roles, I/O, and owned graph objects

Each agent reads/writes a slice of shared state. "Owns" = sole writer of that object type.

Agent	Input	Output	Owns (sole writer)
Orchestrator	run trigger, blackboard	routing decisions, confidence gate verdicts	run state, control flow
Ingestor/Resolver	RawReport[]	resolved Signal[] bound to CDG FKs	Signal nodes (§2)
Graph-Builder	Signal[] , static topology	live CDG subgraph + edges	Asset/Service/edge updates (§1)
Correlator	Signal[] , CDG	Incident (members + excluded)	Incident node (§3)
Root-Cause Analyst	Incident , CDG	RootCause + mechanism + evidence	RootCause node (§4)
Cascade Simulator	CDG, RootCause	Risk Index trajectory, time-to-depletion	RiskNode values (§5)
Solution-Generator	RootCause , actuator map	Candidate[] (target/authority/lag)	Candidate objects
Simulator/Validator	Candidate[] , hydraulic twin	per-candidate cascade- halt + net Risk delta (provenance='sim')	SimResult objects
Adversarial Critic / Red-Team	RootCause , top Candidate[]	Refutation[] or PASS	Refutation objects
Ranker/Selector	surviving Candidate[]	THE Solution + ranked alternatives + lineage	Solution object
Decision- Gate/Auditor	Solution	Recommendation → human gate (G03)	Decision record
Memory/Learning	closed Decision + outcome	recalibrated priors, ECE	calibration store (G04)

4.2 Topology & coordination

Orchestrator-led pipeline with an **adversarial verify loop**. The happy path is linear; the Critic injects two back-edges (RC refute → re-analyze; Solution refute / no valid candidate → re-generate).



Shared state = CDG + blackboard. The CDG (kuzu/rustworkx, §1) is durable canonical truth; the blackboard is a per-run scratch dict keyed by object id, holding in-flight artifacts and the `confidence` ledger. Agents communicate *only* by reading/writing these — no direct messaging, so any agent can be re-run idempotently.

```

blackboard = {
    "incident": INC, "root_cause": RC, "candidates": [...],
    "sim_results": {...}, "refutations": [...],
    "confidence": 0.0, "loop": {"rc": 0, "sol": 0},
}

```

Loop-until-confident control (Orchestrator):

```

def gate(bb):
    if bb["refutations"]:
        target = "root_cause" if any(r.kind=="RC" for r in
        bb["refutations"]) \
        else "solution"
        bb["loop"][target[:3]] += 1
        return ("regen_rc" if target=="root_cause" else "regen_sol")
    if not any(c.valid for c in bb["candidates"]):
        return "regen_sol"
    if bb["confidence"] >= THRESH: return "ship"
    if bb["loop"]["sol"] >= MAX:
        (G13)
        return "abstain"
        / inspect"
    return "regen_sol"

```

The Critic is the teeth of the rubric (§D-rubric): it asserts symptom-only candidates (ER staffing, 911 call-takers), demands a `SimResult` showing net Risk Index improvement across **all** affected nodes, and forces abstention rather than a confident wrong answer.

4.3 Framework mapping

Recommend LangGraph — the pipeline-with-back-edges is literally a `StateGraph`: nodes = agents, the typed `State` = blackboard, conditional edges = the `gate()` router, and built-in checkpointing gives durable resume + the audit trail the Decision-Gate needs. **ruv-swarm / claude-flow is already in this environment** (`mcp__ruflo__swarm_init`, `agent_spawn`, `coordination_orchestrate`, `hive-mind_consensus`) and is the drop-in swarm runtime when you want spawned worker agents + a shared `agentdb` blackboard with built-in consensus for the Critic vote. **CrewAI** (role/task abstraction) or **AutoGen** (conversational critic loop) are viable alternatives but need hand-rolled state + back-edges. Whichever runtime, agents call the §1–§6 engine contracts as tools and ground answers via Haystack/LlamaIndex (§6, G26).

4.4 Zarga run — swarm trace

```
t0 Orchestrator: swarm_init(topology=pipeline); spawn 11 roles
t1 Ingestor: S1..S6 → resolved Signals (S6 low src_confidence)
t2 Graph-Builder: bind S1→PS-12, S2→WATER-ZN, S3→HOSP-ZN-1, S4→JUNC-7,
S5→PSAP-911
t3 Correlator: CDG-reachability → INC-ZN-WATER = {S1..S5}; EXCLUDE S6 (no
path) —handoff→ RC
t4 Root-Cause Analyst: backward traversal → apex PIPE-ZN-44; Layer-B KB →
"corrosion+pressure transient, 1998 main, overdue insp"; RC-1 conf 0.80
t5 Critic (RC): "is HOSP-ZN-1 an independent cause?" → check: HOSP is
descendant of WATER-ZN → REFUTED-AS-CAUSE → PASS on RC ✓
t6 Cascade Sim: Zarga Risk 31.7→54.3 attributed to PIPE-ZN-44; R-3 time-
to-depletion = deadline
t7 Solution-Gen: C1 close isolation valve + WAJ repair crew→PIPE-ZN-44; C2
reroute alt main→re-pressurize PS-12; C3 tanker convoy→R-3 (stopgap, Civil
Defense); C4* surge ER (symptom)
t8 Validator: sim(provenance='sim'): C1 PS-12 1.1→6.0 bar, Risk→32.1 ✓;
C2 ✓ but de-pressurizes adjacent zone (+harm); C3 alone Risk→48 (symptom-
relief only); C4* no cascade change
t9 Critic (Sol): KILL C4* (acts on symptom node); FLAG C2 (second-order
harm unbounded); C3 not root-resolving alone
t10 Ranker: Solution = C1 (durable, net Risk min) PAIRED with C3
(stopgap beats deadline); alts: C2, C3-only; conf 0.80 ×
evidence_completeness; lineage Insight→RC-1→INC-ZN-WATER→S1..S5
t11 Decision-Gate: Recommendation → HUMAN AUTH (named WAJ role) before live
valve-close (G03); stays provenance='sim' until approved (G14)
t12 Memory: on outcome, log expected-vs-actual Risk delta →
recalibrate (ECE)
```

The trace shows the discriminating moves: S6 excluded at t3, the loud symptoms (S3/S5) demoted below PIPE-ZN-44 at t4–t5, the symptom-only candidate C4 killed at t9, and the fast stopgap (C3) paired with—not substituted for—the durable fix (C1) at t10. See §3 for the engine internals each agent invokes and §5–§6 for the rubric gates and lineage objects the Ranker and Decision-Gate enforce.

5. Verified Open-Source Tech Stack

The Water-Security Crisis Brain is assembled entirely from mature, actively maintained open-source projects, each mapped to a specific stage of the Zarga cascade loop: detect a spike, correlate signals into one incident, build/score a causal dependency graph, localize the root cause, simulate candidate interventions against ground-truth water physics, and optimize the response. The tables below group every verified repo by brain component, with the single best pick per category in **bold**. Star counts are approximate and license/maintenance notes reflect the verification snapshot. A few repos are flagged "caution" (stale or restrictive licensing) but are retained because they remain technically sound options; the recommended minimal stack at the end avoids them.

Graph databases & in-memory graph engines

Repo	~Stars	License	Maps to brain component	Why pick
networkx/networkx	17k	BSD-3-Clause	Crisis Dependency Graph store + traversal substrate	Reference in-memory typed DiGraph/MultiDiGraph; <code>ancestors()</code> / <code>descendants()</code> drive backward causal traversal and weighted shortest-path finds symptom-to-root paths. Ubiquitous, zero-friction default.
Qiskit/rustworkx	1.7k	Apache-2.0	Faster traversal substrate for same graph store	Rust core, same conceptual ancestors/descendants/shortest-path ops, fast enough for repeated what-if cascade traversals (API is distinct, so code must be ported, not swapped).
memgraph/memgraph	4.1k	BSL 1.1 (source-available) + MEL	Persistent server-grade graph store	Cypher, ACID, MAGE algorithms, native vector + text indexes co-locate signal embeddings next to the graph; for when the graph outgrows one Python process.
cozodb/cozo (caution)	4.0k	MPL-2.0	Embedded persistent graph store	Single-file durable store with Datalog recursion, HNSW vectors, and time-travel (query graph at incident-onset vs now); stale (last release Dec 2023), pre-1.0.

Entity resolution / record linkage

Repo	~Stars	License	Maps to brain component	Why pick
dedupeio/dedupe	4.5k	MIT	Canonicalize multi-agency identifiers	Active-learning matcher needs little labeled data; Gazetteer/RecordLink modes match incoming agency signals against the canonical asset registry (the graph's node table).
moj-analytical-services/splink	2.2k	MIT	Resolve agency references to canonical asset/service/location	Fellegi-Sunter scorer + blocking at ~1M records/min on DuckDB; outputs same-entity probabilities (e.g. PIPE-ZN-44 vs "trunk main 44"). Scales to Spark/Athena.
J535D165/recordlinkage	1.0k	BSD-3-Clause	Fine-grained custom resolution pipeline	pandas-native with geo (haversine)/string/date comparators and built-in precision/recall metrics to validate the resolver.

Stream processing / complex-event correlation

Repo	~Stars	License	Maps to brain component	Why pick
pathwaycom/pathway	63.2k	BSL 1.1 → Apache-2.0	Stitch lagged, out-of-order signals into one incident	Incremental Rust engine with native as-of/interval temporal joins correlates a late hospital-strain signal back to the earlier pressure-drop window without reprocessing; ideal for lagged dependency edges.
quixio/quix-streams	1.6k	Apache-2.0	Kafka-based windowed correlation	RocksDB-backed stateful tumbling/hopping/sliding windows; group-by keyed by asset/zone correlates S1..S5 into one incident object.
faust-streaming/faust	1.9k	BSD-3-Clause	Lightweight Kafka agent + windowed Tables	Async "agents" accumulate per-signal contributions in windowed Tables; fits bursty inputs like the +320% 911 spike.
bytewax/bytewax (caution)	2.0k	Apache-2.0	Event-time windowed correlation	Python+Rust event-time windows/watermarks correlate signals despite late arrival; technically strong but stale/at-risk (core team stepped back ~May 2025).

Anomaly & spike detection

Repo	~Stars	License	Maps to brain component	Why pick
yzhao062/pyod	9.9k	BSD-2-Clause	Batch multivariate spike detector	60+ detectors, uniform sklearn-style API; <code>decision_function</code> gives a continuous severity score per signal that fires incident-spike candidates into the correlation loop.
online-ml/river	5.8k	BSD-3-Clause	Streaming/online detection path	O(1)/constant-memory <code>learn_one / predict_one</code> scoring flags a spike the instant PS-12 pressure collapses; drift detectors catch regime changes (sustained reservoir depletion).
arundo/adtk (caution)	1.2k	MPL-2.0	Interpretable rule-based spike detection	Level-shift/persist/seasonal detectors give explainable evidence (6.2→1.1 bar drop, R-3 depletion, 911 surge); effectively unmaintained since 2020.
linkedin/luminol (caution)	1.2k	Apache-2.0	Spike detection + first-pass correlation	<code>correlate()</code> ranks which signals co-move and with what lag, rejecting S6 noise; dormant since 2023.

Causal inference & root-cause analysis

Repo	~Stars	License	Maps to brain component	Why pick
py-why/dowhy	8.1k	MIT	Core root-cause attribution engine	Feed the dependency DAG to DoWhy-GCM; <code>attribute_anomalies</code> ranks each node's contribution so PIPE-ZN-44 outranks loud 911/ER symptoms, plus <code>distribution_change</code> and a refutation/falsification API for the VALID-SOLUTION confidence checks.
py-why/causal-learn	1.6k	MIT	Causal graph discovery/validation	PC/FCI/GES/LiNGAM/Grange learn or validate edges/lags from agency time-series when the graph is incomplete; FCI's latent-confounder handling flags S6 noise as non-causal.
phamquillan/RCAEval	149	MIT	RCA validation/benchmark harness	Scores the engine's cause-ranking (is PIPE-ZN-44 top-1?) against 15 published baselines (BARO, CIRCA, RCD, ϵ -Diagnosis...) on labeled cascades; swappable RCA implementations.
salesforce/PyRCA (caution)	555	BSD-3-Clause	Turnkey RCA localization + dashboard	Bayesian-inference/random-walk topology localizers map ~1:1 onto walking the graph backward from symptoms; dormant ~2.5 yrs.

Graph ML / GNN / network analysis

Repo	~Stars	License	Maps to brain component
pyg-team/pytorch_geometric	23.8k	MIT	Learned propagation weights + embeddings
benedekrozemberczki/pytorch_geometric_temporal	3.0k	MIT	Time-lagged cascade forecasting
dmlc/dgl	14.3k	Apache-2.0	Scalable/heterogeneous propagation & embeddings

Simulation: system-dynamics / agent-based / discrete-event

Repo	~Stars	License	Maps to brain component	Why pick
projectmesa/mesa	3.7k	Apache-2.0	Agent-based cascade simulation (with/without intervention)	PS-12, R-3, WATER-ZN, HOSP-ZN-1, PSAP-911 and the population become interacting agents; NetworkGrid maps onto the dependency graph, mesa-geo adds GIS, and WITH-vs-WITHOUT runs measure whether the cascade halts and NRI drops.
SDXorg/pysd	451	MIT	System-dynamics stocks/flows	Model reservoir volume, service coverage, tanker queue as stocks/flows; inject an intervention as a parameter change (restore 1.1→6.2 bar) and integrate forward to prove the fix over time.
salabim/salabim	393	MIT	Discrete-event, resource-constrained response	Tankers compete for the JUNC-7 point (limited resource), 911 calls queue at PSAP-911, repair crews dispatch with lag; KPIs (wait, queue, time-to-restore) with/without intervention. Zero dependencies.
transentis/bptk_py	32	MIT	Hybrid SD+ABM scenario harness	Define baseline + multiple intervention scenarios (repair, reroute, tanker surge) in one framework, diff outputs to rank by NRI reduction; hybrid SD+ABM keeps hydraulics and responder agents in one model.

Water-distribution & infrastructure modeling (domain-specific)

Repo	~Stars	License	Maps to brain component	Why pick
USEPA/WNTR	433	BSD-3-Clause	Ground-truth water-physics simulator + solution validator	Zarqa maps ~1:1 (PIPE-ZN-44=pipe, PS-12=pump, R-3=tank, WATER-ZN=junctions); a leak/break event yields the pressure-dependent-demand pressure drop and resilience metrics, and re-running with a candidate fix VALIDATES whether service recovers. <code>wn.to_graph()</code> plugs into the dependency graph.
OpenWaterAnalytics/EPANET	388	MIT	Low-level reference hydraulic solver	Canonical <code>.inp</code> format and the C toolkit that actually computes node pressures/pipe flows; raw, fast, scriptable evaluations inside the intervention-ranking loop and to validate WNTR.
OpenWaterAnalytics/EPyT	74	EUPL-1.2	Programmable Python bridge to EPANET	400+ get/set functions let solver agents mutate the model (close a valve, change diameter/demand) and immediately re-simulate; tight enumerate-and-test loop against the authoritative engine.

Optimization / resource allocation / network flow

Repo	~Stars	License	Maps to brain component	Why pick
google/or-tools	13.5k	Apache-2.0	Intervention-selection optimizer	CP-SAT picks the discrete intervention bundle (valves, repairs, tanker counts) under resource/authority/budget/time constraints to maximize NRI reduction; built-in min-cost-flow/VRP route tankers and re-supply across the network.

Recommended minimal stack

The smallest set that builds the full Zarqa loop end-to-end is: **NetworkX** as the in-memory Crisis Dependency Graph store and traversal substrate; **dedupe** for entity resolution, canonicalizing messy multi-agency identifiers into the graph's node table; **Pathway** for stream-processing correlation, using temporal as-of joins to stitch the lagged, out-of-order signals into a single incident; **DoWhy** (DoWhy-GCM) as the causal/RCA engine, feeding the graph in as a DAG and using `attribute_anomalies` plus its refutation API to rank PIPE-ZN-44 as the root cause with confidence; **WNTR** (over the EPANET engine) as the domain-specific water-physics simulator that both reproduces the cascade and validates that candidate interventions restore pressure/service; and an **agent framework — Mesa** — to run WITH-vs-WITHOUT intervention simulations of the interacting assets, services, and population and confirm the cascade halts and the National Risk Index drops. This six-repo core (all permissive BSD/MIT except Pathway's BSL-to-Apache license) covers graph store, entity resolution, correlation, causal/RCA, water-domain simulation, and agent-based what-if execution; OR-Tools is the natural seventh addition once intervention selection must be optimized rather than enumerated.

6. Build Plan & MVP

Dependency-ordered build of the Zarqa brain. Each step names its **component category** (the §5 Tech Stack picks the exact repo) and the **acceptance gate** that proves it works on the §0 fixture. Engines are wired through a shared `CDG` handle and the canonical objects from §1–§6.

6.1 Dependency-ordered sequence

Step 1 — Graph store + canonical model. Stand up the typed CDG and persist the Zarqa subtree `PIPE-ZN-44 → PS-12 → R-3 → WATER-ZN → {HOSP-ZN-1, JUNC-7, PSAP-911, POP-ZN}` with edge weights + lags. *Category:* graph engine + embedded graph DB (§5). *Gate:* `subgraphWithin(WATER-ZN, k=2)` returns all symptoms; `ancestors(WATER-ZN)` includes `PIPE-ZN-44`.

```
node = {"id": "PIPE-ZN-44", "type": "Asset", "material": "ductile-iron",
        "dia_mm": 600, "installed": 1998, "inspection": "overdue"}
edge = {"src": "PS-12", "dst": "R-3", "rel": "supplies", "w": 0.9, "lag_min": 12}
```

Step 2 — Ingestion + entity resolution. `RawReport → Signal` bound to CDG foreign keys, event-time watermarking. *Category:* API/validation + spatial + streaming (§5). *Gate:* `S1..S6` ingest; `S1` resolves to `PS-12`, `S3` to `HOSP-ZN-1`; each `Signal.asset_ref` is a live CDG id.

Step 3 — Correlation / incident stitching. 4-dimension link scoring with CDG-reachability decisive; connected-component emission. *Category:* correlation engine over the graph (§5). *Gate (T-CORR):* `INC-ZN-WATER = {S1..S5}`, `S6` excluded as noise.

Step 4 — Root-cause. Backward traversal (Layer A apex) + intra-asset failure-mode KB (Layer B mechanism). *Category:* causal-inference lib (§5). *Gate (T-RC):* emits `RC-1 = PIPE-`

ZN-44 (conf 0.80), ranked strictly above HOSP-ZN-1 / PSAP-911 ; mechanism = corrosion + pressure transient.

Step 5 — Cascade / Risk Index. Local risk + lag-gated forward propagation, roll-up to governorate. *Category:* same graph engine + risk operator (§5). *Gate (T-RISK):* Zarqa index 31.7 → 54.3 attributed to PIPE-ZN-44 ; WATER-ZN =68.9.

Step 6 — Intervention engine. From RC-1 , enumerate the action menu over the *controllability map* (act on cause's controllers, not symptom nodes): isolation/reroute valve, WAJ repair crew, tanker stopgap to R-3, PS-12 pressure-management. Each tagged {target, authority, resources, lag, cost} . Feasibility/authority + cost gates demote infeasible candidates. *Category:* candidate generator + routing/allocation solver (§5).

```
cands = generate(RC-1, controllability_map)          # §6 intervention
          generator
cands = [c for c in cands if feasible(c, ledger, authority_map)]
```

Gate: every candidate targets a root-cause-side node; an ER-staffing action is auto-flagged symptom-only and rejected.

Step 7 — Simulation-based validation. Apply each candidate in provenance='sim' (G14): hydraulic twin for PS-12 pressure/flow + adjacent-zone effects, stock-and-flow for R-3 time-to-depletion, §5 cascade operator for Risk delta; second-order/counterfactual harm re-propagation. *Category:* hydraulic twin + system-dynamics + counterfactual lib (§5). *Gate (T-RISK + T-ISO):* valve-close+reroute restores PS-12 toward 6.2 bar, drives index 54.3 → ~31.7, net harm across all nodes improves; sim never moves the live index.

Step 8 — Solver swarm wrapper. Wrap Steps 2–7 as cooperating roles (Ingestor, Correlator, RC-Detective, Simulationist, Generator, Feasibility, Red-Team, Ranker, Decision-Gate) over a blackboard on the shared CDG; ranker selects THE solution + alternatives; via-negativa floor defaults to ABSTAIN and routes to the human authorization gate (G03/G13). *Category:* agent orchestration + state-machine/durable-workflow + source-cited grounding (§5). *Gate:* end-to-end run on §0 fixture returns ranked solution with calibrated confidence + lineage Insight→RC-1→INC-ZN-WATER→S1..S5 ; every assertion cites a CDG object.

Step 9 — Thin cockpit / insight surface. Read-only brief: incident, root cause, cascade trajectory, the chosen intervention vs runner-ups, authorization button. *Category:* source-cited narration over the canonical model (§5). *Gate:* every displayed number is read from the CDG (not generated); "unknown" shown on insufficient evidence.

6.2 MVP cut-line

In: one end-to-end loop — ingest S1..S5 → stitch INC-ZN-WATER → RC-1 → cascade 54.3 → generate+simulate the valve-close+reroute (with tanker stopgap) → rank → present + human gate, passing T-CORR/T-RC/T-RISK/T-ISO on the sealed Zarqa fixture (and ≥1 red-team holdout from the §0 calibration set, closing G02).

Deferred (post-MVP): live-execution crossover and Temporal-backed decision/outcome/feedback loop (G04 recalibration); full EPANET .inp beyond the Zarqa

zone; multi-incident concurrency; the other five crisis domains; confidence recalibration via reliability diagram/ECE at scale.

6.3 Milestones

M	Deliverable	Steps	Acceptance gate
M0	CDG + canonical model loaded	1	traversal primitives return Zarqa subtree
M1	Signals → incident	2–3	T-CORR (S1..S5 in, S6 out)
M2	Diagnosis	4–5	T-RC (RC-1) + T-RISK (54.3)
M3	Validated solution	6–7	sim halts cascade, bounds harm, T-ISO
M4	Swarm + cockpit + holdout	8–9	full loop + human gate + 1 holdout pass
